Олимпиада Я - Профессионал
Программная инженерия
Финал
Задание 1

1. Введение

Ссылка на репозиторий с решением: https://github.com/arprspb/y2026f

Цель работы — реализовать автоматизированную систему транскрибации голосовых команд оператора с использованием open-source ASR (VOSK), сохранением аудио и результатов в базе данных, веб-интерфейсом с прослушиванием записей, а также аутентификацией, ролями и сценариями из задания (запись команды, проверка и подтверждение, поиск по истории, администрирование пользователей).

Краткое описание решения. Клиентская часть на Vue 3 и Vite отображает экран входа, запись с микрофона с предпросмотром распознавания, историю с фильтрами, карточку записи с аудио и правками, страницу пользователей для администратора. Серверная часть на FastAPI принимает загрузку аудио, конвертирует в формат WAV 16 кГц моно через ffmpeg, распознаёт речь моделью VOSK из каталога, задаваемого переменной окружения; из текста извлекаются ключевая фраза команды из фиксированного набора и идентификатор (цифры, буквы или смесь). Данные хранятся в PostgreSQL (SQLAlchemy 2, миграции Alembic). Первичный администратор создаётся при старте из переменных окружения; саморегистрация отключена. Роли: администратор, оператор записи, оператор проверки; доступ к операциям разграничен на уровне API.

Ключевые каталоги репозитория для ориентирования:

backend

frontend

READMEю.md с запуском и моделью VOSK:

2. Описание технического решения

2.1. Схема решения

Логические компоненты:

- веб-браузер с SPA на Vue 3, маршрутизация и состояние авторизации (Pinia)
- обратный прокси при сборке в Docker (nginx) для статики UI и проксирования префикса API к сервису FastAPI
- сервис API на Uvicorn, маршруты REST под префиксом api
- подсистема ASR: библиотека vosk, модель на диске, путь из настроек
- подсистема парсинга текста команд: нормализация устных числительных и склейка фрагментов, извлечение команды и идентификатора
- PostgreSQL для пользователей и записей голосовых команд
- файловое хранилище для загруженных аудиофайлов (каталог uploads)

Браузер отправляет аудио на эндпоинт предпросмотра, API сохраняет временный файл, конвертирует в WAV, вызывает VOSK, возвращает транскрипт и разбор; после подтверждения пользователем файл переносится в постоянное хранилище, в БД создаётся запись с метаданными. Оператор проверки получает список записей и может править текст и подтверждать.

Детализация шагов предпросмотра и сохранения (логический сценарий без привязки к конкретным именам функций):

1. Клиент записывает звук с микрофона и отправляет бинарный поток на API предпросмотра вместе с токеном авторизации.
2. Сервер сохраняет вход во временный файл, вызывает конвертацию в WAV моно 16 кГц при необходимости.
3. Сервер вызывает распознавание VOSK по готовому WAV и получает строку текста.
4. Сервер передаёт текст в парсер команд и получает ключевую фразу, основной идентификатор и список найденных фрагментов.
5. Ответ API содержит транскрипт, поля разбора и при необходимости служебные поля для отладки; клиент показывает результат пользователю до окончательного сохранения.
6. При явном подтверждении сохранения клиент вызывает эндпоинт подтверждения; сервер переносит файл в постоянный каталог uploads, формирует уникальное имя файла, записывает строку в voice_commands с привязкой к текущему пользователю и заполненными полями raw_transcript, parsed_command, parsed_identifier.

Детализация проверки и подтверждения:

1. Пользователь с ролью проверки запрашивает список записей с фильтрами; сервер применяет правила доступа из role_access и зависимостей с текущим пользователем.
2. В карточке записи клиент запрашивает поток аудио по идентификатору записи; сервер отдаёт файл из хранилища при наличии прав.

При редактировании текста и нажатии подтверждения сервер обновляет edited_transcript, выставляет confirmed, confirmed_at, confirmed_by_user_id.

2.2. Технологии и обоснование

- Python 3 и FastAPI — быстрая разработка типизированного API, автодокументация OpenAPI, асинхронные эндпоинты для ввода-вывода с файлами и БД
- PostgreSQL — надёжное хранение связей пользователь—записи и метаданных подтверждения
- SQLAlchemy 2 и Alembic — декларативные модели и версионирование схемы
- Vue 3 и Vue Router — одностраничное приложение без перезагрузки при смене экранов
- axios — HTTP-клиент с передачей JWT в заголовке Authorization
- VOSK — офлайн-распознавание речи на русском без внешнего облачного API, соответствие требованию open-source ASR
- ffmpeg — приведение произвольного входного аудио к формату, ожидаемому VOSK
- bcrypt и JWT (python-jose) — хранение пароля и выдача токена после входа

Настройки окружения и путей:

В файле .env.example лежит пример переменных окружения, его необходимо скопировать в файл .env, и установить нужные настройки.

2.3. База данных: инфологическая и даталогическая модель

Инфологическая модель

Сущность «Пользователь»

Назначение: учётная запись для входа в систему и привязки действий к конкретному лицу.

Атрибуты на уровне предметной области:

- внутренний номер записи (суррогатный ключ для связей)
- логин (уникальное имя для входа)
- секрет аутентификации (пароль не хранится в открытом виде, в системе — только хэш)
- роль в системе: администратор; оператор, создающий голосовые записи; оператор, проверяющий и подтверждающий результаты
- признак того, что учётная запись разрешена к входу (блокировка при нарушениях или по решению администратора)
- момент появления записи в базе (для аудита и сортировки)

Связи: один пользователь может быть автором многих голосовых команд; один пользователь может подтвердить многие чужие голосовые команды (если у него роль проверки). Удаление пользователя-автора приводит к удалению всех его записей команд; удаление пользователя-подтвердившего не удаляет сами команды, а только обнуляет ссылку на подтверждающего.

Сущность «Голосовая команда»

Назначение: одна зафиксированная попытка: что было сказано, что распознано, что сохранено в файле, как извлечены команда и идентификатор, кто подтвердил результат.

Атрибуты на уровне предметной области:

- связь с пользователем-автором (кто говорил в микрофон)
- имя файла с аудио во внешнем хранилище (каталог uploads на сервере)
- текст, полученный после ASR и нормализации для парсера
- текст после ручной правки оператором проверки (если правка была)
- извлечённая ключевая фраза команды из условия задания (если найдена)
- извлечённый идентификатор (цифровой или смешанный; основной для отображения — первый найденный)
- время записи дорожки (метаданные момента сохранения)
- признак того, что результат проверен и принят
- время подтверждения (если подтверждение выполнено)
- связь с пользователем, который подтвердил запись (если применимо)

Даталогическая модель

СУБД: PostgreSQL. Типы дат — с часовым поясом (timestamptz). Роли пользователя хранятся как перечисление на стороне БД (enum userrole), значения в коде совпадают с литералами в Python.

Таблица users

Имя в БД: users. Назначение: хранение учётных записей.

Поля:

- id — целое, первичный ключ, автоинкремент
- username — строка до 64 символов, уникальная, с индексом для быстрого поиска по логину
- hashed_password — строка до 255 символов, хэш пароля (алгоритм на стороне приложения)

- role — тип userrole, не пусто; по умолчанию в приложении при создании пользователя без явной роли может выставляться оператор записи (см. код моделей)
- is_active — логическое, по умолчанию истина; ложь означает блокировку входа
- created_at — время с часовым поясом, на сервере БД по умолчанию текущее время (func.now)

Ограничения и индексы: уникальность username; первичный ключ по id.

Таблица voice_commands

Имя в БД: voice_commands. Назначение: одна строка на одну сохранённую голосовую команду с аудио и текстами.

Поля:

- id — целое, первичный ключ, автоинкремент
- user_id — внешний ключ на users.id, не пусто, с индексом; при удалении пользователя-автора — каскадное удаление строки команды (ON DELETE CASCADE)
- audio_filename — строка до 512 символов, относительное или итоговое имя файла в хранилище
- raw_transcript — текст без ограничения длины на уровне ORM (Text), не пусто по умолчанию может быть пустой строкой; сырой результат цепочки ASR и нормализации
- edited_transcript — текст, допускается NULL; правка оператором проверки поверх распознавания
- parsed_command — строка до 255 символов или NULL; нормализованная ключевая фраза
- parsed_identifier — строка до 255 символов или NULL; основной извлечённый идентификатор
- recorded_at — время с часовым поясом, по умолчанию на сервере текущее время
- confirmed — логическое, по умолчанию ложь
- confirmed_at — время или NULL
- confirmed_by_user_id — целое или NULL, внешний ключ на users.id; при удалении подтвердившего пользователя — SET NULL (ON DELETE SET NULL), индекс для выборок по подтверждающему

Связи ORM: user — автор записи; confirmed_by_user — кто подтвердил (опционально).

Перечисление ролей (enum)

Значения в коде приложения (Python) и в БД согласованы:

admin — полный доступ к управлению пользователями
operator_record — создание голосовых записей, просмотр своих записей в рамках правил API
operator_verify — просмотр записей (в т.ч. для проверки), правка транскрипта, подтверждение

Файл где содержатся перечисления backend/app/enums.py

Реализация таблиц в SQLAlchemy:

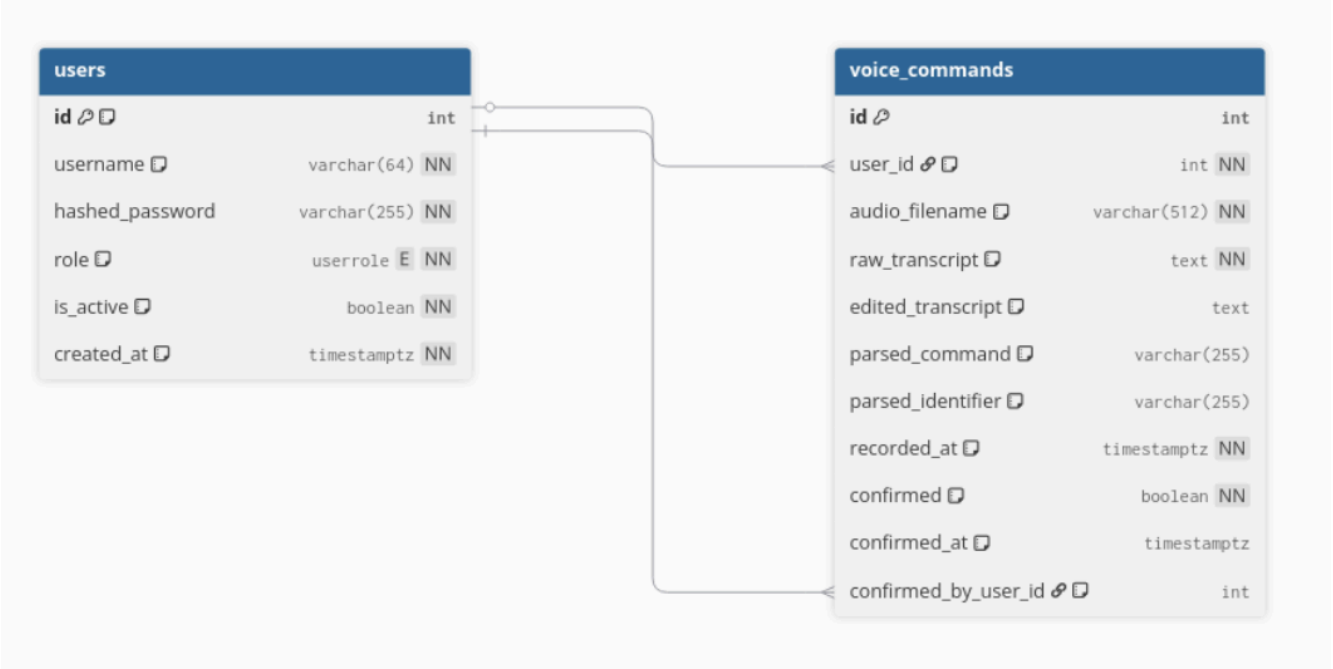
backend/app/models.py

Миграции Alembic с появлением ролей и поля подтверждающего пользователя:

backend/alembic/versions/001_initial.py

backend/alembic/versions/002_roles_and_confirmed_by.py]

Диаграмма:



Ключевые атрибуты: для пользователя — уникальный логин и роль; для голосовой команды — связь с автором, файлы аудио и распознавания, извлечённые команда и идентификатор, блок подтверждения (флаг, время, кто подтвердил). Связи один-ко-многим от users к voice_commands по

авторству и отдельно по подтверждению отражены внешними ключами и политиками удаления выше.

2.4. Процессы, сценарии использования и информационные потоки

Соответствие сценариям MVP из задания

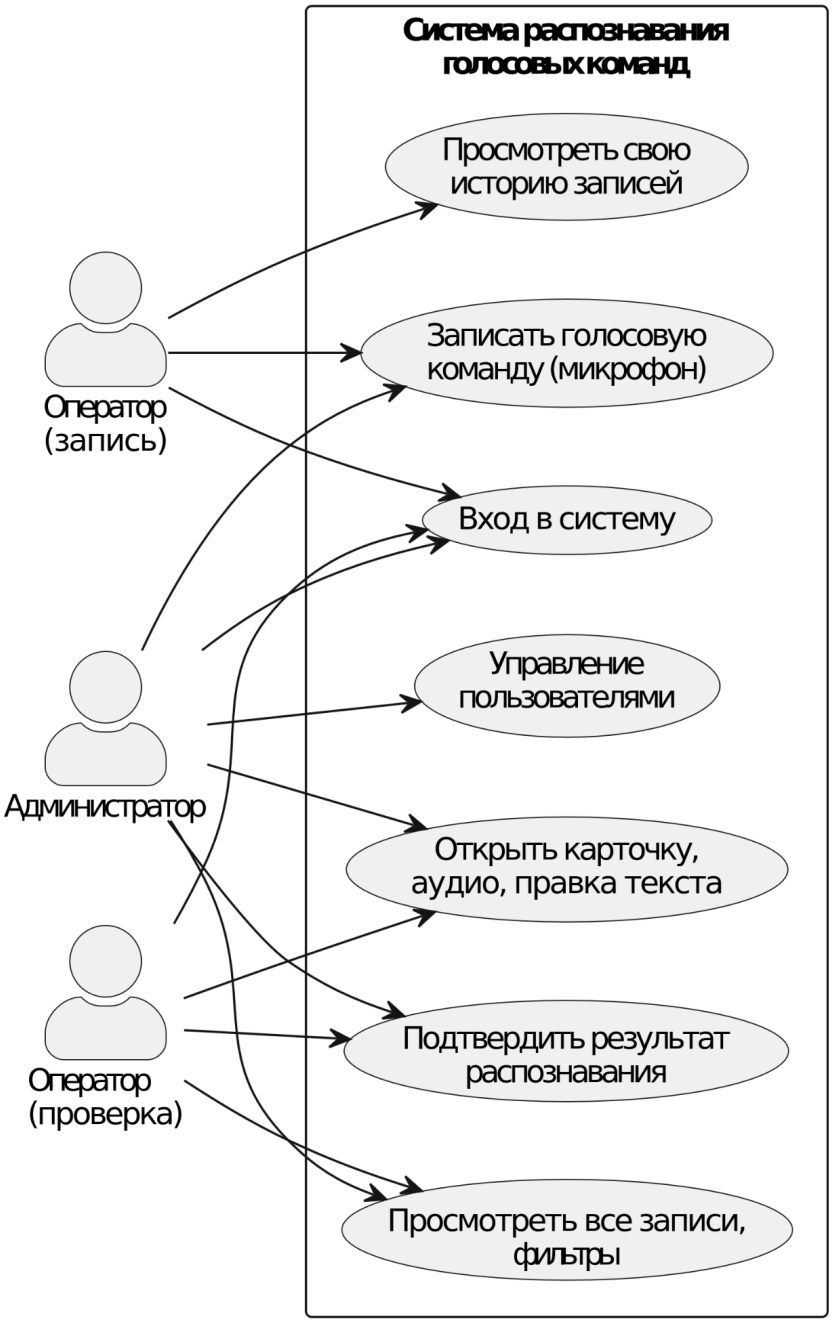
Сценарий 1. Оператор распознаёт команду: вход в систему, экран записи, озвучивание, предпросмотр распознавания, сохранение в истории — реализовано экраном записи и цепочкой API preview и confirm.

Сценарий 2. Корректировка результата: оператор проверки открывает карточку из истории, прослушивает аудио, правит текст при необходимости, подтверждает — реализовано экраном детализации и полем подтверждения в базе.

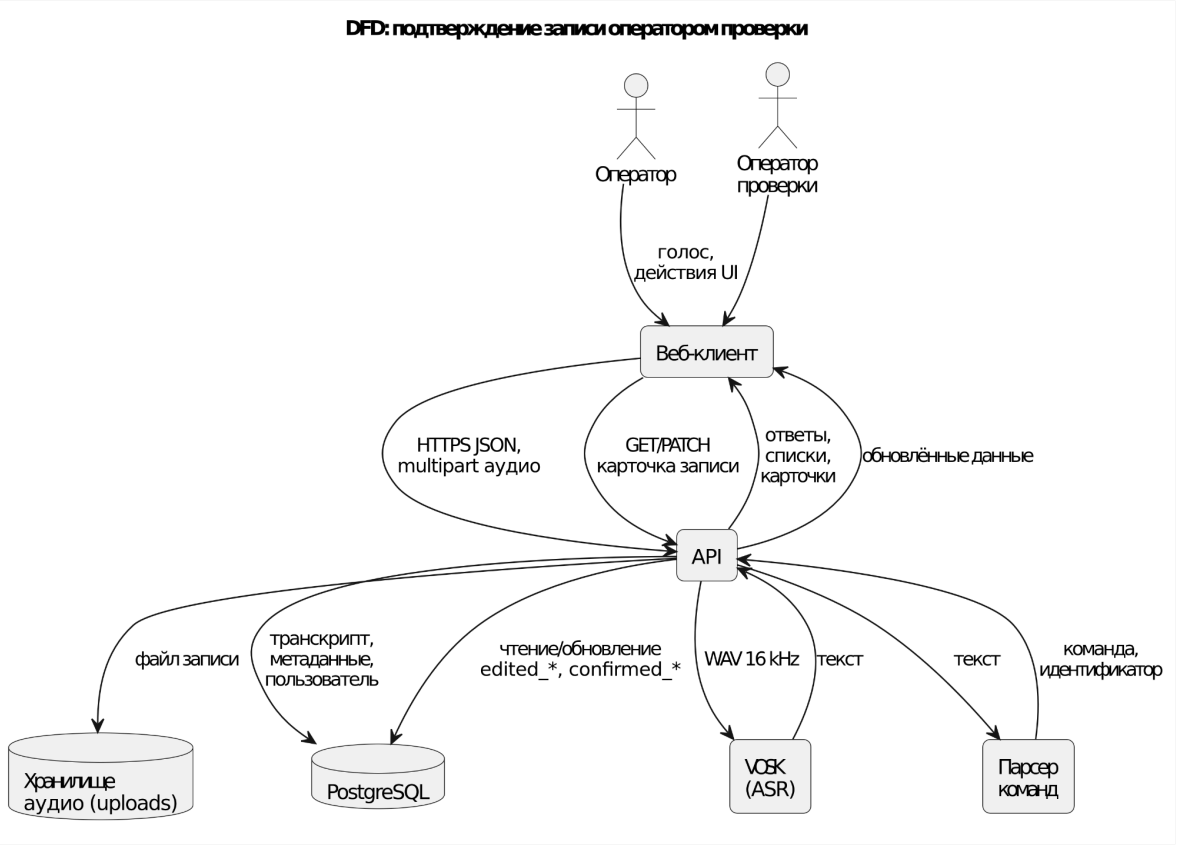
Сценарий 3. Поиск и верификация: список с фильтрами по команде, параметру, дате, оператору — экран истории и запросы к API.

Сценарий 4. Администрирование: создание пользователей с ролью, блокировка — экран «Пользователи» для роли администратор, эндпоинты создания и обновления пользователя.

Диаграмма прецедентов (UML Use Case):



Диаграммы потоков данных (DFD) для уровня «голос — распознавание — сохранение» и отдельно для подтверждения:



Маршруты API голосовых команд и пользователей:

backend/app/routers/voice_commands.py

backend/app/routers/users.py

Правила доступа по ролям для записи, просмотра всех записей и подтверждения:

backend/app/role_access.py

backend/app/deps.py

2.5 Реализация, алгоритмы, тестирование, оценка качества ASR

Общая цепочка обработки записи голоса

На уровне логики приложения цепочка выглядит так: входной аудиофайл приводится к WAV 16 кГц моно 16 бит; по WAV вызывается VOSK;

полученная строка проходит нормализацию для команд; из нормализованного текста извлекаются ключевая фраза и идентификатор; результаты сохраняются в базе и отдаются клиенту. Ниже даны псевдокод и детали по модулям.

Псевдокод: распознавание речи (ASR) по файлу WAV

Вход: путь к файлу в формате PCM 16-bit mono 16 kHz.

Шаги:

-
- Загрузить модель VOSK один раз (singleton), путь к каталогу модели взять из настроек; при первом обращении проверить наличие структуры модели (например am/final.mdl), при необходимости «поднять» вложенную папку из архива (resolve_vosk_model_dir).
- Создать KaldiRecognizer с частотой дискретизации из WAV.
- В цикле читать фрагменты сэмплов из WAV; для каждого фрагмента вызывать AcceptWaveform; при промежуточном результате с непустым text добавлять фрагмент к списку строк.
- После EOF вызвать FinalResult и добавить финальный text при наличии.
- Склеить все части через пробел и обрезать краевые пробелы.

Псевдокод отражает код в:

backend/app/services/asr.py

Конвертация произвольного аудио в WAV выполняется отдельно (ffmpeg), см.:

backend/app/services/audio_convert.py

Псевдокод: нормализация транскрипта перед извлечением команды

Цель: привести устные числительные к цифрам и склеить последовательности одноразрядных цифр в один токен (чтобы восьмизначный номер не оставался восемью отдельными словами), затем при необходимости склеить блок цифр с «разнесённым» буквенным хвостом, как часто отдаёт ASR.

Функция normalize_transcript_for_commands(raw):

1. Применить normalize_text: нижний регистр, замена буквы ё на е, схлопывание пробелов.
2. Разбить строку на токены по пробелам.
3. Каждый токен, если он совпадает со словарём русских числительных «ноль»…«девять», заменить на одну цифру.

4. Пройти токены слева направо: если подряд идут односимвольные цифры, объединить их в один токен без пробелов; иначе оставить токен как есть.
5. Склеить токены пробелами.
6. Применить итеративную подстановку по шаблону «длинный блок цифр + пробел + хвост из букв/цифр»: если в хвосте есть цифра, склеить цифры и хвост в один токен; если хвост из нескольких однобуквенных слов подряд, тоже склеить с цифровым блоком (код вроде «в б д» после номера); иначе оставить как есть. Повторять до стабилизации.

Псевдокод: извлечение ключевой команды

Функция extract_command(normalized):

1. Иметь упорядоченный список фраз команд от более длинных к более коротким: отменить регистрацию, отменить обработку, начать обработку, завершить обработку, зарегистрировать.
2. Для каждой фразы в этом порядке: если фраза входит как подстрока в normalized, вернуть эту фразу.
3. Иначе вернуть отсутствие команды.

Псевдокод: извлечение идентификаторов

Функция extract_identifiers(normalized, command):

1. Если command найдена, взять подстроку после первого вхождения фразы команды (хвост); если команда есть и хвост пустой, вернуть пустой список идентификаторов.
2. Иначе искать по всей строке normalized.
3. На области поиска применить несколько классов регулярных выражений: смешанные буквенно-цифровые токены; шаблон с лидирующими цифрами и буквенным продолжением; только цифры как отдельные «пробеги»; только буквы длиной от трёх символов с фильтром стоп-слов (номер, плавки, обработку и т.д.).
4. Собрать пары (позиция, токен), отсортировать по позиции, убрать дубликаты токенов с сохранением порядва первого появления.
5. Вернуть список строк.

Псевдокод: полный разбор текста после микрофона

Функция parse_voice_text(raw):

1. prepared := normalize_transcript_for_commands(raw)
2. normalized := normalize_text(prepared)
3. cmd := extract_command(normalized)
4. ids := extract_identifiers(normalized, cmd)

5. primary := первый элемент ids или отсутствие
6. вернуть структуру с полями command, identifier (primary), identifiers_found

Реализация на Python с регулярными выражениями и константами фраз:

backend/app/services/command_parser.py

Создание начального администратора при старте приложения:

backend/app/bootstrap.py

Тестирование

Автоматические тесты на уровне парсера команд и правил ролей без поднятия полного веб-стека:

backend/tests/test_command_parser.py

backend/tests/test_role_helpers.py

Запуск: из каталога backend с установленными зависимостями выполнить pytest tests/.

Точность и скорость распознавания речи

Критерий оценивания задания включает точность и скорость на примерах команд из условия. Фиксированные численные показатели в отчёт без проведения измерений на контрольном наборе не вносятся. Рекомендуемая методика для воспроизводимой оценки:

1. закрепить версию модели VOSK и параметры ffmpeg
2. подготовить набор тестовых фраз из задания и близких к ним формулировок
3. для каждой фразы записать аудио в одинаковых условиях микрофона
4. засечь время от начала запроса до ответа API с финальным транскриптом на том же оборудовании, где будет проверка
5. сравнить эталонный текст с распознанным по символьной ошибке или по совпадению извлечённой команды и идентификатора

По умолчанию в инструкции к развёртыванию указана крупная русская модель vosk-model-ru-0.42 для повышения качества по сравнению с компактной small-ru; при ограниченных ресурсах можно указать путь к малой модели в переменной окружения. Субъективно при использовании крупной модели на сервере качество распознавания коротких команд с цифрами и буквами выше, чем у малой модели, ценой большего размера на диске и времени загрузки модели в память.

3. Демонстрация работы

Инструкция пользователя и видеодемонстрация

По условию задания в состав решения входит развёрнутая инструкция пользователя с учётом ролей. Кратко: вход по логину и паролю (выдаёт администратор); оператор записи работает с экраном «Запись», предпросмотром и сохранением в историю; оператор проверки пользуется «Историей» и карточкой записи для правки текста и подтверждения; администратор дополнительно открывает «Пользователи» для создания учётных записей с ролью оператора записи или проверки, смены роли и блокировки. Пункт «Запись» в меню не показывается пользователям только с ролью проверки.

Полная пошаговая инструкция по экранам и кнопкам для каждой роли вынесена в приложение:

appendices/user-manual-by-role.txt

Видеоинструкция и видео-демонстрация работы:
video.mp4

4. Инструкция для развёртывания у эксперта

Ниже — минимальная последовательность действий в терминале после клонирования репозитория на машину с установленными Docker и Docker Compose. Команды можно копировать по строкам. Пояснения, смена секретов, локальный запуск без Docker, облегчённая модель VOSK и типичные неполадки вынесены в приложение.

Предусловия: свободны порты 5432, 8000 и 8080 на хосте (или заранее измените проброс в docker-compose.yml).

Клонирование (один раз):

```
git clone https://github.com/arprspb/y2026f.git
cd y2026f
```

В случае запуска на большей модели - скачивание и распаковка модели VOSK (из корня репозитория):

```
mkdir -p models
cd models
```

```
wget https://alphacephei.com/vosk/models/vosk-model-ru-0.42.zip
unzip vosk-model-ru-0.42.zip
cd ..
```

Проверка: должен существовать файл models/vosk-model-ru-0.42/am/final.mdl. Если структура архива дала лишнюю вложенность, перенесите папки так, чтобы am лежала прямо внутри vosk-model-ru-0.42.

Запуск всех сервисов (из корня репозитория):

```
docker compose up --build
```

Дождаться в логах успешных миграций Alembic и запуска Uvicorn в контейнере api.

Проверка API во втором терминале:

```
curl -s http://127.0.0.1:8000/health
```

В ответе ожидается JSON с полем status равным ok.

Открыть в браузере:

- интерфейс: http://localhost:8080
- Swagger: http://localhost:8000/docs

Вход: логин и пароль начального администратора задаются в .env файле (BOOTSTRAP_ADMIN_USERNAME, BOOTSTRAP_ADMIN_PASSWORD); в поставляемой конфигурации по умолчанию admin и admin. Перед публичной демонстрацией рекомендуется сменить пароль и секреты, см. приложение.

Развёрнутое руководство по развёртыванию с дублированием команд, переменными окружения, вариантом без Docker и тестами:

appendices/deployment-for-expert.txt

Краткий перечень маршрутов API:

appendices/api-overview.txt

Конфигурация для справки:

docker-compose.yml

.env.example

7. Заключение

Реализована система распознавания голосовых команд с использованием VOSK и open-source стека: веб-клиент на Vue, сервер на FastAPI, PostgreSQL для данных, разграничение ролей администратора и двух типов операторов, сохранение аудио и метаданных подтверждения. Парсер извлекает команды из условия задания и цифро-буквенные идентификаторы. Предусмотрены инструкция развёртывания, схема базы в формате DBML, диаграммы прецедентов и потоков данных в PlantUML, автоматические тесты парсера и правил ролей.

Соответствие требованиям MVP из задания: форма входа, запись с микрофона, ASR, извлечение команды и идентификатора, сохранение файла и распознавания в БД с временем и пользователем и подтверждением, отображение в интерфейсе, управление пользователями с ролями. Сценарии записи, проверки с подтверждением, поиска по истории и администрирования пользователей покрыты функционалом и могут быть продемонстрированы по файлу шагов для видео.

Дальнейшее развитие возможно в сторону формальных замеров WER на фиксированном наборе фраз, нагрузочного тестирования API и доработки фильтров истории под предметную область.

Для решения задачи использовалась IDE Cursor с применением генеративного ИИ в режимах Plan и Agent. Промпты, использованные для решения задачи лежат в файле промпты.txt.

Также для решения задачи использовались промпты в виде так называемых скиллов. Их можно найти в директории .agents/skills (файлы SKILL.md в поддиректориях каждого скилла).

В том числе, там расположен файл .agents/skills/olympiad-report/SKILL.md, который был создан с помощью IDE Cursor. Промпты, использованные для создания этого скилла, находятся в файле создание_скилла.txt.